TEMPERATURE SENSOR EXERCISE

1.0 Learning Outcomes.....1
2.0 Hardware Set-Up.....1
3.0 Simulink Set-Up and Temperature Sensor Testing.....2
4.0 Low-Pass Filter and Testing.....3
5.0 Exercises.....4
6.0 Concluding Remarks.....4
Recommended Citation.....5

1.0 Learning Outcomes

Upon completing this practical exercise, students should be able to demonstrate the Learning Outcomes presented in Table 1. At the end of the exercise, revisit each Learning Outcome and confirm that it has been achieved.

Table 1: *Learning Outcomes for the Practical Exercise*

ID	Learning Outcome	Achieved
1	Explain and demonstrate how to acquire temperature measurements using an analogue temperature sensor.	☐
2	Use a component datasheet to configure an analogue sensor for accurate temperature measurement.	☐
3	Implement a low-pass filter to reduce measurement noise and improve signal quality.	☐

2.0 Hardware Set-Up

The exercise involves connecting a temperature sensor to an Arduino Uno analogue input and calibrating it using the sensor datasheet to measure temperature accurately, as shown in Figure 1.

Note: While the ACE-Box (Base + Sense) can be used for this exercise, it is not required. Only the individual components listed below are needed.

Required hardware

- ATmega328P microcontroller (Arduino Uno)
- USB Type-B cable
- Breadboard
- TMP36 low-voltage temperature sensor
- 3 × male-to-male breadboard wires

TMP36 Temperature Sensor Specifications

- The TMP36 is a low-voltage analogue temperature sensor manufactured by Analog Devices.
- It operates from a supply voltage (terminal 1) of 2.7 V to 5.5 V and measures temperatures from −40 °C to +125 °C.

- The analogue output (terminal 2) is connected to an Arduino analogue input pin, and terminal 3 is ground.
- The TMP36 datasheet can be found here.

Hardware Assembly Steps

1. Connect the analogue output of the TMP36 (terminal 2) to an analogue input pin on the Arduino Uno (A4 is used in the example circuit).
2. Connect terminal 1 of the TMP36 to the 5 V supply pin on the Arduino Uno.
3. Connect terminal 3 of the TMP36 (GND) to a GND pin on the Arduino Uno.

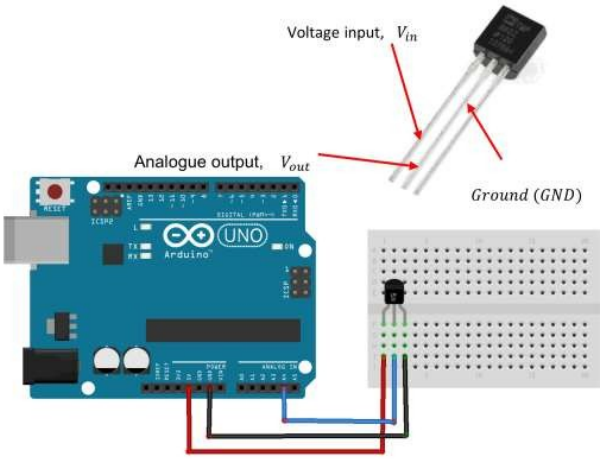


Figure 1: *Hardware Set-Up for the Temperature Sensor Exercise*

3.0 Simulink Set-Up and Temperature Sensor Testing

The complete Simulink model for the hardware described above is shown in Figure 2, and is explained step by step below.

Simulink Model Construction Steps

1. In the Analog Input block, set the Sample time to 0.01 s and the Pin number to A4.
2. The output from Pin A4 is a 10-bit value that ranges from 0 to 1023. For example, if the reading is 150, then 150/1023 = 0.1466, a ratio between 0 and 1. Multiplying by the 5 V supply from the Arduino gives 0.7330 V. In Simulink, this is done with a Gain block of 5/1023.
3. The TMP36 output includes a built-in 0.5 V offset so that it can measure temperatures below 0 °C (see the datasheet). To remove this offset, add a Sum block and a Constant block set to 0.5, and set the signs of the Sum block to "+−" so that the constant is subtracted from the sensor reading. Using the value from Step 2, the corrected reading becomes 0.7330 V − 0.5 V = 0.2330 V.
4. The corrected voltage can now be converted to temperature in degrees Celsius. The TMP36 has a scale factor of 10 mV/°C, so for every 1 °C change in temperature the output voltage changes by 10 mV. Dividing by 0.01 V, which is the same as multiplying by 100 using a Gain block, gives the temperature: 0.2330 V × 100 = 23.3 °C.
5. Deploy the model and observe the temperature, in degrees Celsius, of the area around the sensor. Notice that the measured signal is noisy, which

makes it difficult to read a reliable value. Would this signal be useful for reading the temperature of a living room?

To recap how to run Simulink code generation and deploy the model to the Arduino Uno, see Section 4.0 of the LED (Digital) Exercise or visit Code Generation.

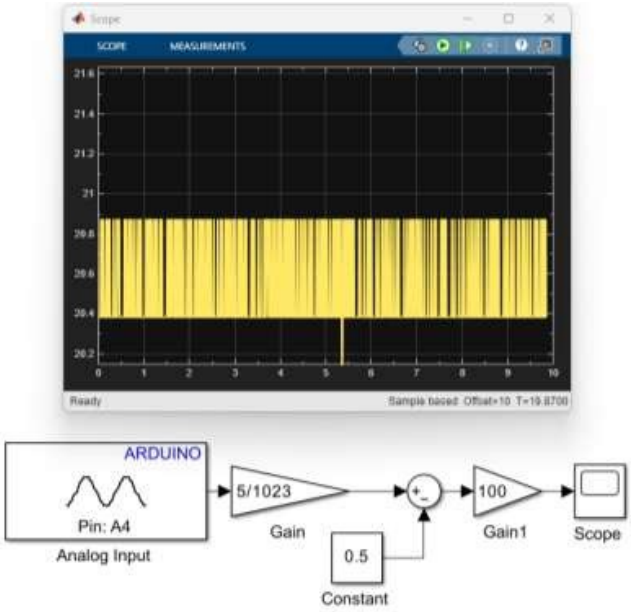


Figure 2: *Simulink Set-Up for the Temperature Sensor Exercise*

4.0 Low-Pass Filter and Testing

A low-pass filter is now added to the Simulink model. The filter attenuates high-frequency signals, such as measurement noise, and allows low-frequency signals, such as the underlying change in temperature, to pass through.

Low-Pass Filter Set-Up Steps

1. Add a Discrete Filter block as shown in Figure 3. Open its block parameters and set the numerator and denominator coefficients to those shown in Figure 3. The filter has been designed for a sample time of 0.01 s, the same sample time used elsewhere in the model, and must only be used with this sample time.
2. Add a Mux block so that the filtered and unfiltered signals can be compared on the same Scope.
3. Deploy the model and compare the filtered and unfiltered signals. Observe how the low-pass filter removes the high-frequency noise and makes the signal smoother. This gives a temperature measurement that is suitable for use in control.

The Discrete Filter block implements the filter as a difference equation, which is evaluated once every sample.

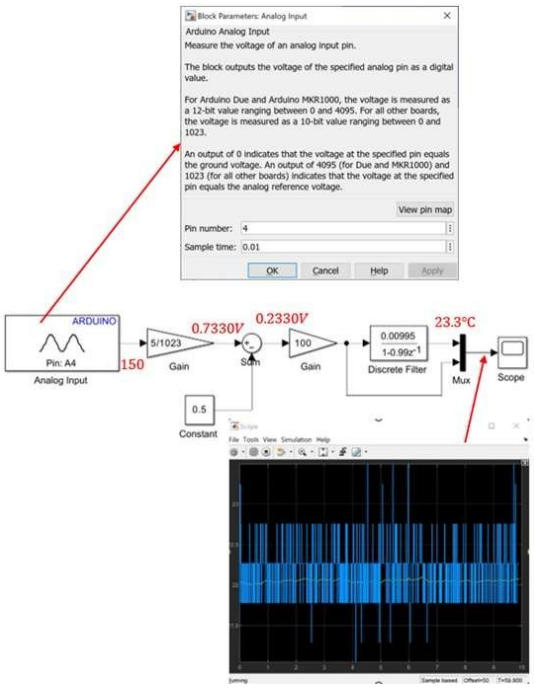


Figure 3: *Low-Pass Filter Simulink Model and Testing*

5.0 Exercises

1. Warm the sensor slightly (e.g., by holding it) and observe the dynamic response of both the filtered and unfiltered signals.
2. Add simple logic to switch on an LED when the temperature exceeds a defined threshold (e.g., 30 °C).
3. Modify the Simulink model so that the measured temperature is output in degrees Fahrenheit.

6.0 Concluding Remarks

This exercise has demonstrated how to acquire and calibrate analogue temperature measurements using the TMP36 sensor and its datasheet. You implemented voltage scaling, offset compensation and unit conversion to obtain the temperature in degrees Celsius.

The introduction of a discrete low-pass filter highlighted the importance of signal conditioning in practical measurement systems. Raw sensor signals often contain noise, and filtering improves measurement stability and usability for control applications.

The complete signal chain, sensor → ADC → calibration → filtering → output, represents a fundamental embedded systems workflow and provides a strong foundation for temperature monitoring and closed-loop thermal control.

Before completing the exercise, return to the Learning Outcomes in Table 1 and confirm that you are confident you can demonstrate each one. If so, tick the corresponding Achieved box.

Recommended Citation

If you have used this practical exercise in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). Temperature Sensor Exercise. ACE-Lab. Teaching Notes: ACE-Lab Practical Exercises, Version 1.0. Zenodo. https://doi.org/10.5281/zenodo.23885018